

UNIVERSIDAD RAFAEL LANDÍVAR

FACULTAD DE INGENIERÍA

LICENCIATURA EN INGENIERIA EN INFORMATICA Y SISTEMAS

MANEJO E IMPLEMENTACIÓN DE ARCHIVOS

PROYECTO 1 (Manual de Usuario)

NOMBRE:

Marvin Antonio Vásquez Chán

CARNET:

1558525

QUETZALTENANGO, SEPTIEMBRE DE 2026

CAMPUS SAN ALBERTO HURTADO, S. J

Manual Técnico

Aplicación de Escritorio con Gestión de Configuración de Usuario

1. Introducción

El presente proyecto desarrolla una aplicación de escritorio en Python orientada al manejo robusto de archivos mediante un sistema de configuración de usuario. La interfaz gráfica fue construida con PyQt6 y funciona como medio para modificar y visualizar las preferencias, mientras que la persistencia de los datos constituye la parte central de la solución.

La configuración se almacena en formato JSON y contiene información relacionada con el nombre de usuario, tema de interfaz, idioma, tamaño de fuente, colores y foto de perfil. La aplicación implementa lectura, validación, actualización y protección de la configuración, incluyendo escritura mediante archivo temporal y conservación de un respaldo previo.

Además, se incorporan mecanismos para responder de forma controlada ante archivos inexistentes, archivos JSON corruptos, estructuras inválidas y problemas de permisos, evitando que la aplicación finalice con errores no controlados. La implementación utiliza UTF-8 para conservar correctamente tildes, ñ y otros caracteres propios del idioma español.

2. Objetivos

2.1. Objetivo general

Desarrollar una aplicación de escritorio en Python que permita gestionar de forma segura una configuración de usuario mediante archivos JSON, integrando una interfaz gráfica en PyQt6 y mecanismos de persistencia, respaldo, validación y manejo de errores.

2.2. Objetivos específicos

- Implementar la lectura y escritura de la configuración utilizando JSON y codificación UTF-8.
- Desarrollar una ventana de Settings que permita modificar todos los parámetros solicitados.
- Implementar escritura segura mediante un archivo temporal y conservar un respaldo de la configuración anterior.
- Controlar los casos de archivo ausente, archivo corrupto o con formato inválido y falta de permisos.
- Comprobar mediante pruebas que la información permanece disponible al cerrar y volver a iniciar la aplicación.

3. Descripción general de la solución

La aplicación inicia cargando la configuración existente. Cuando el archivo no está disponible, se utilizan valores predeterminados. Una vez cargada la configuración, la ventana principal presenta un menú con las opciones Archivo, Edición, Ver y Settings. Las tres primeras opciones son simuladas, mientras que Settings es funcional y permite modificar la configuración del usuario.

Al confirmar los cambios, la interfaz envía la nueva configuración al gestor de archivos. Este conserva el estado anterior en un respaldo, escribe la nueva información en un archivo temporal y finalmente reemplaza el archivo principal. De esta forma, la interfaz y la persistencia se mantienen separadas y la lógica de archivos puede probarse independientemente.

Flujo general

Inicio → cargar configuración → validar → mostrar interfaz → modificar Settings → guardar de forma segura → actualizar configuración.

4. Tecnologías utilizadas

Tecnología	Uso en el proyecto	Motivo
Python	Lenguaje principal	Permite implementar la lógica de archivos y la aplicación de escritorio.
PyQt6	Interfaz gráfica	Proporciona ventanas, menús, selectores de colores y selector de archivos.
JSON	Persistencia	Representa la configuración de forma estructurada y legible.
os	Operaciones de archivos	Permite comprobar existencia y reemplazar el archivo de forma controlada.
shutil	Copia de foto	Permite copiar la imagen seleccionada a la carpeta del proyecto.

5. Selección del formato de almacenamiento

5.1. JSON como formato elegido

Se seleccionó JSON porque la información del proyecto representa propiedades de configuración de distinta naturaleza dentro de una misma estructura: texto, valores numéricos, colores, opciones de selección y una ruta de imagen. JSON permite expresar esas propiedades con pares clave-valor de forma clara y fácil de procesar desde Python.

Además, el módulo json de Python permite convertir directamente entre la representación almacenada y los diccionarios utilizados por la aplicación. Esto reduce la complejidad de la lógica de persistencia y facilita validar que cada propiedad requerida esté presente.

5.2. Comparación con CSV

Como alternativa se consideró CSV. Este formato es apropiado para conjuntos de datos tabulares, donde cada fila representa un registro con columnas definidas. Sin embargo, la configuración del proyecto no representa registros repetitivos; corresponde a un conjunto de propiedades heterogéneas, por lo que CSV resulta menos natural para este caso.

Criterio	JSON	CSV
Tamaño	Generalmente compacto para configuraciones pequeñas.	También compacto, especialmente en tablas.
Legibilidad	Alta; las claves identifican directamente cada propiedad.	Alta en datos tabulares, menor para configuraciones heterogéneas.
Facilidad de parseo	Soporte directo mediante el módulo json de Python.	Soporte disponible, pero requiere tratar filas y columnas.
Tipos de datos	Puede representar texto, números, booleanos, listas y objetos.	Normalmente se interpreta como valores tabulares que deben convertirse.
Estructura	Permite objetos y estructuras anidadas.	Principalmente filas y columnas.
Adecuación al proyecto	Alta para una configuración de usuario.	Menor para esta estructura de preferencias.

La robustez frente a corrupción no depende únicamente del formato. En esta solución se complementa JSON con validación, manejo de JSON inválido, recuperación desde el respaldo y valores predeterminados, de acuerdo con el comportamiento solicitado por el laboratorio.

6. Estructura del archivo de configuración

El archivo config.json contiene las propiedades utilizadas por la aplicación. Un ejemplo representativo es el siguiente:

```
{  
  "nombre_usuario": "Marvin Vásquez",  
  "tema_interfaz": "oscuro",
```

```
"idioma": "es-ES",  
  
"tamaño_fuente": 12,  
  
"color_barra_menu": "000000",  
  
"color_letra": "#FFFFFF",  
  
"foto_perfil":  
"images\\perfil.jpg" }
```

Las siete propiedades corresponden directamente a los parámetros que el laboratorio solicita configurar desde Settings. La validación comprueba la presencia de estos campos y, cuando corresponde, restringe sus valores a las opciones permitidas por el proyecto.

7. Arquitectura y organización del proyecto

La organización divide la aplicación en un archivo de entrada, un módulo de persistencia y módulos de interfaz. Esta separación facilita localizar los errores y explicar el proyecto durante la defensa.

```
Proyecto_Archivos/  
|  
|— main.py  
|— config_manager.py  
|— config.json  
|— .gitignore  
|  
|— ui/  
  
| |— __init__.py  
| |— main_window.py  
| |— settings_window.py  
  
|  
|— images/  
  
| |— perfil.jpg
```



Responsabilidad de los módulos

Módulo	Responsabilidad
main.py	Inicia QApplication y muestra MainWindow.
config_manager.py	Carga, valida, guarda y recupera la configuración.
main_window.py	Crea la ventana principal, menús y aplica la configuración visual.
settings_window.py	Permite editar las preferencias, elegir colores y seleccionar la foto.
config.json	Conserva la configuración actual.
config.bak	Conserva un ejemplo del respaldo de una configuración anterior.
config.tmp	Archivo temporal de escritura; no es permanente.

8. Implementación de la persistencia

8.1. Valores predeterminados

Cuando la configuración todavía no existe o no puede recuperarse, se utiliza una estructura de valores predeterminados. Esto evita que el programa dependa de que el usuario haya ejecutado previamente la aplicación.

```
CONFIGURACION_DEFAULT = {
    "nombre_usuario": "Usuario",
    "tema_interfaz": "oscuro",
    "idioma": "es-ES",
    "tamaño_fuente": 12,
```

```
"color_barra_menu": "#000000",  
  
"color_letra": "#FFFFFF",  
  
"foto_perfil": ""  
}
```

8.2. Carga y validación

La función de carga comprueba primero si config.json existe. Cuando no existe, retorna los valores predeterminados. Si existe, se abre con UTF-8 y se utiliza json.load() para interpretar el contenido. Después se ejecuta la validación para comprobar que se encuentren todas las propiedades requeridas y que sus tipos y opciones sean válidos.

```
with open(ARCHIVO_CONFIG, "r", encoding="utf-8") as  
archivo:    configuracion = json.load(archivo)  
  
if not validar_configuracion(configuracion):  
  
    return recuperar_configuracion()
```

8.3. Escritura segura

La escritura no se realiza directamente sobre config.json. Primero se conserva el contenido anterior como respaldo, después se genera el nuevo contenido en config.tmp y finalmente se utiliza

os.replace() para sustituir el archivo final. Este flujo reduce el riesgo de dejar config.json parcialmente escrito si ocurre una interrupción durante la operación.

```
with open(ARCHIVO_TEMPORAL, "w", encoding="utf-8") as archivo:  
  
    json.dump(  
configuracion,  
  
        archivo,  
indent=4,  
  
        ensure_ascii=False  
    )  
  
os.replace(ARCHIVO_TEMPORAL, ARCHIVO_CONFIG)
```

8.4. Respaldo .bak

Antes de reemplazar la configuración actual, se lee el contenido existente de config.json y se copia a config.bak. De esta manera queda disponible el estado anterior y puede utilizarse para recuperación cuando el archivo principal presenta corrupción o estructura inválida.

```
with open(ARCHIVO_CONFIG, "r", encoding="utf-8") as
archivo:

    contenido_anterior = archivo.read()

with open(ARCHIVO_BACKUP, "w", encoding="utf-8") as
archivo:    archivo.write(contenido_anterior)
```

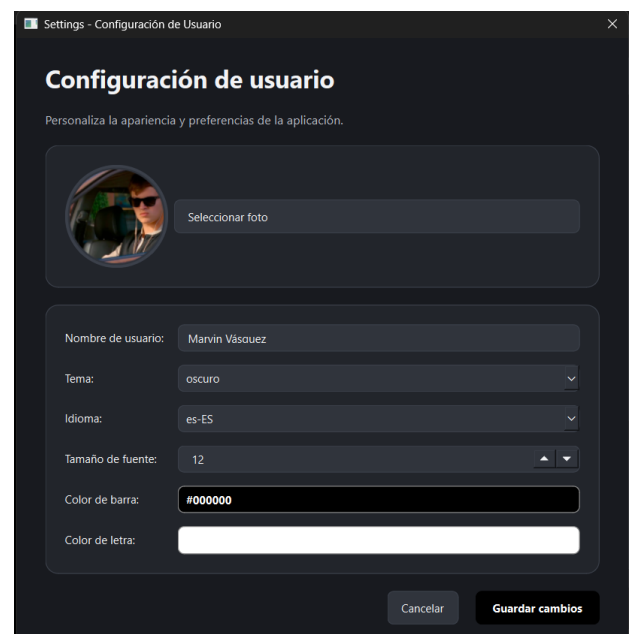
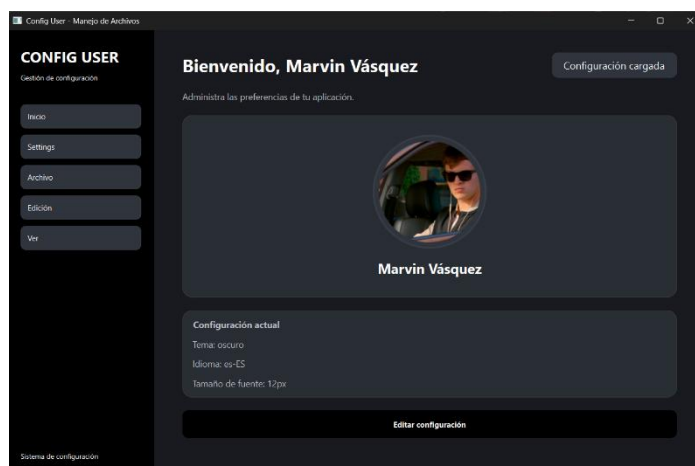
8.5. Archivo temporal .tmp

config.tmp funciona como zona intermedia para la nueva configuración. Una vez terminada la escritura, os.replace() mueve el archivo temporal a la ubicación del archivo definitivo. Al finalizar correctamente el proceso no debe quedar un config.tmp permanente en el proyecto.

9. Interfaz gráfica y Settings

La interfaz se desarrolló con PyQt6. La ventana principal contiene los menús Archivo, Edición, Ver y Settings. De acuerdo con el enunciado, los tres primeros son simulados y Settings es la opción funcional que modifica la configuración persistente.

La ventana Settings incorpora un campo de texto para el nombre, listas desplegables para tema e idioma, un selector numérico para el tamaño de fuente, selectores de color y un selector de archivos para la foto de perfil. Después de guardar, la configuración se aplica nuevamente a la ventana principal.



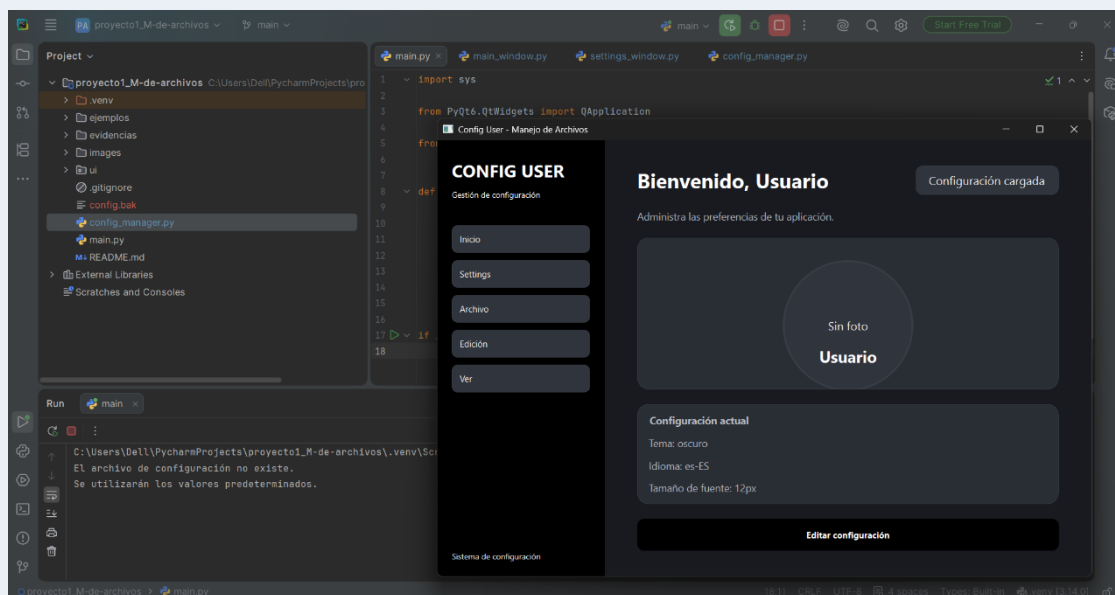
10. Manejo de errores

El sistema utiliza bloques de manejo de excepciones para evitar que errores de lectura o escritura terminen la ejecución sin control. Los casos contemplados son los solicitados por el laboratorio.

10.1. Archivo ausente

Si config.json no existe, la aplicación detecta esta situación con `os.path.exists()`. En lugar de finalizar con un error, utiliza la configuración predeterminada. Este comportamiento permite iniciar la aplicación incluso cuando se trata de la primera ejecución.

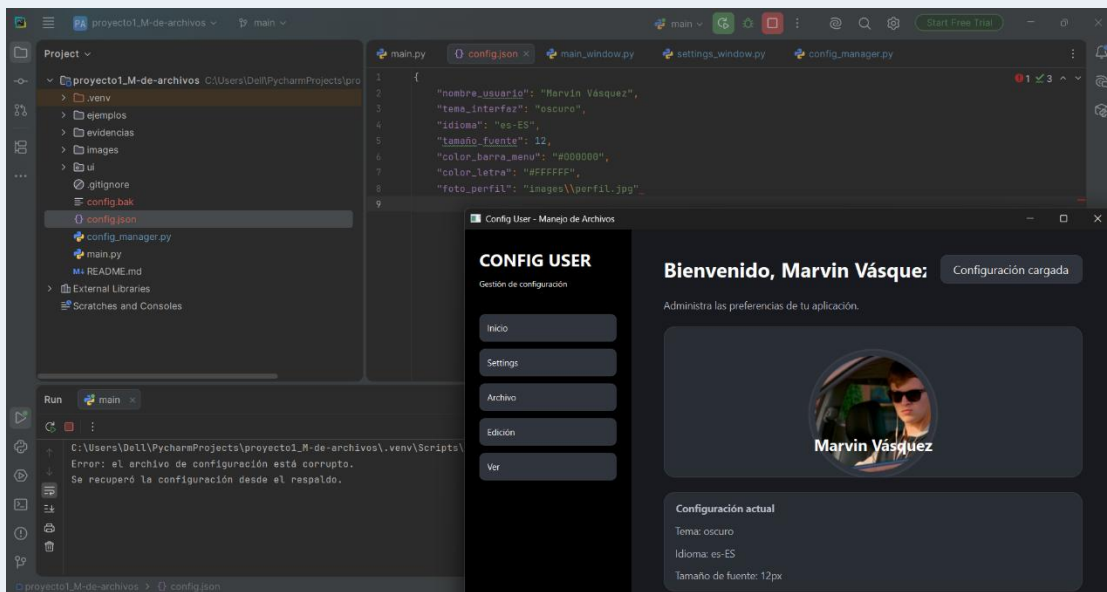
```
if not os.path.exists(ARCHIVO_CONFIG):  
  
    print("El archivo de configuración no  
    existe.")    print("Se utilizarán los valores  
    predeterminados.")    return  
obtener_configuracion_default()
```



10.2. Archivo corrupto

Un archivo corrupto se representa como un JSON que no puede interpretarse, por ejemplo cuando falta una llave o existe una estructura incompleta. `json.load()` genera `json.JSONDecodeError` y la aplicación intenta recuperar la configuración desde `config.bak`.

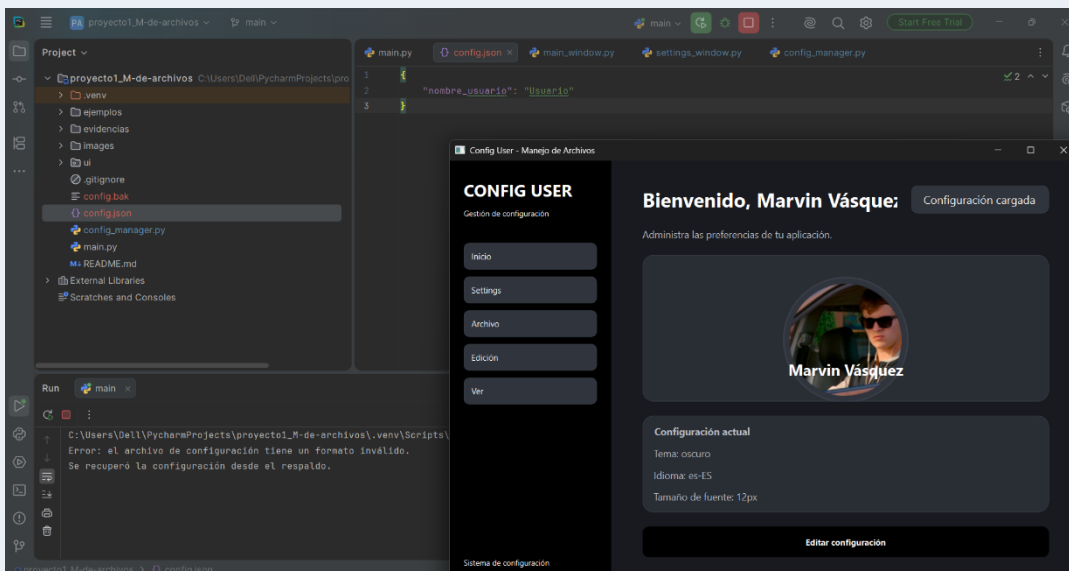
```
except json.JSONDecodeError:  
  
    print("Error: el archivo de configuración está  
    corrupto.")    return recuperar_configuracion()
```



10.3. Formato inválido

El formato puede ser sintácticamente correcto como JSON y aun así ser inválido para la aplicación. Por ejemplo, puede faltar uno de los campos requeridos. La función `validar_configuracion()` identifica esta situación y provoca la recuperación mediante el respaldo o, si no está disponible, el uso de valores predeterminados.

```
if not validar_configuracion(configuracion): print("Error:
el archivo de configuración tiene un formato inválido.")
return recuperar_configuracion()
```

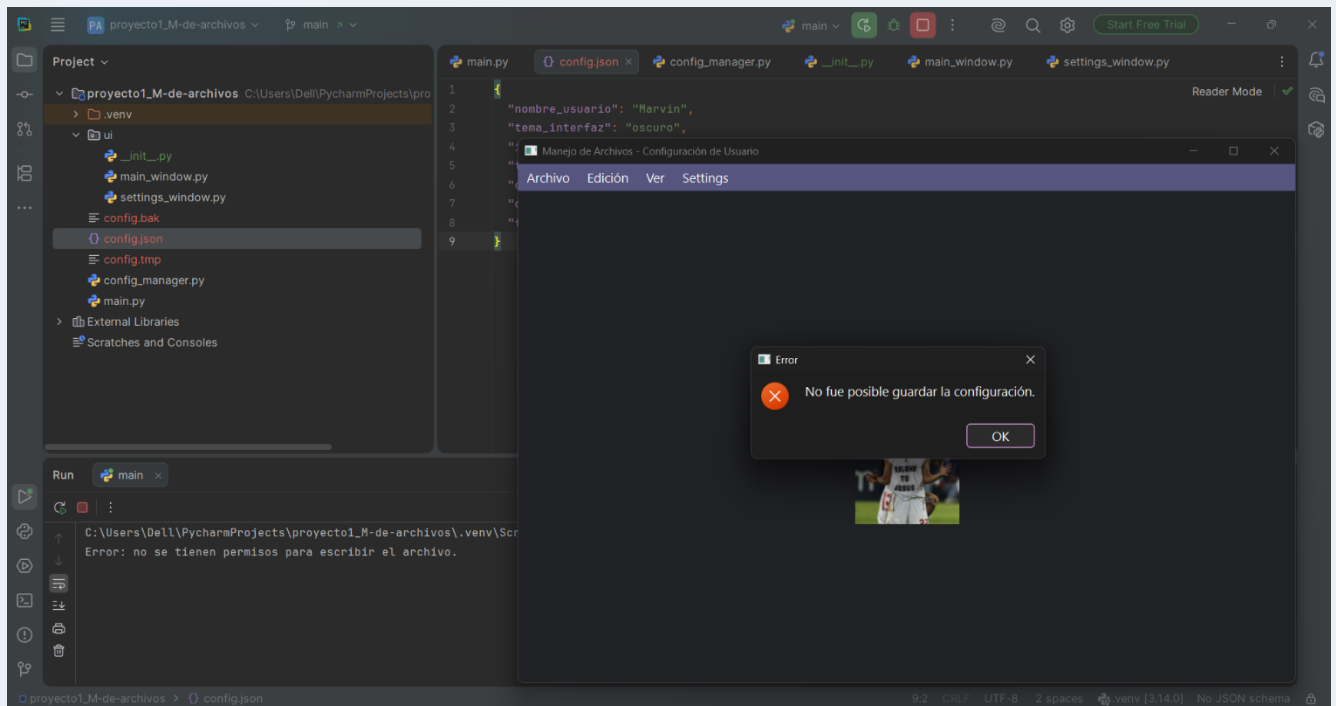


10.4. Falta de permisos

Los errores de acceso se controlan mediante `PermissionError`. Durante la lectura, la aplicación informa que no tiene permisos y utiliza una configuración alternativa. Durante la escritura, informa del problema y evita que el programa termine de forma abrupta.

```
except PermissionError:
```

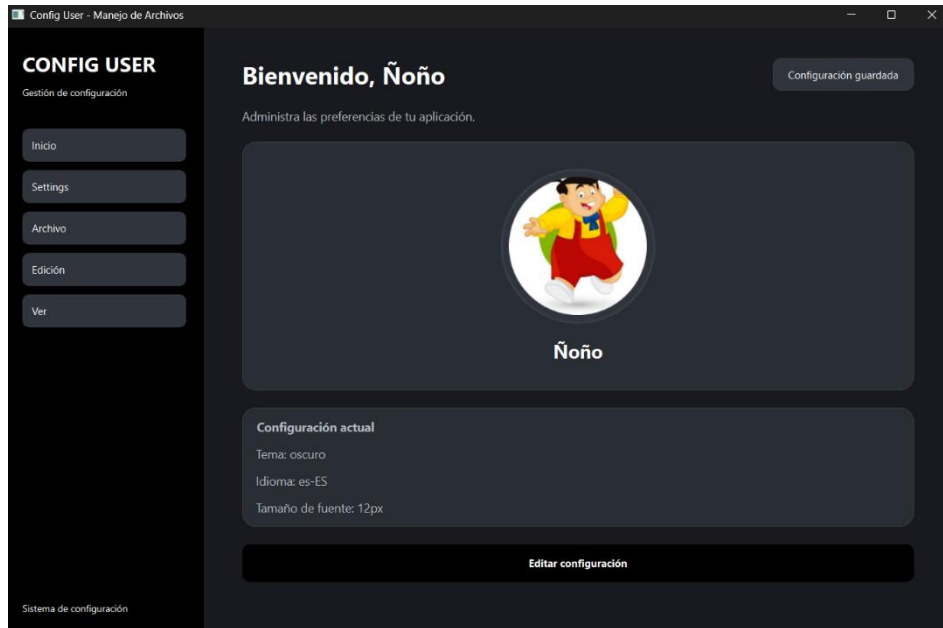
```
    print("Error: no se tienen permisos para leer el archivo.")  
    return obtener_configuracion_default()
```



11. Codificación UTF-8

La lectura y escritura del archivo se realiza especificando `encoding="utf-8"`. En la escritura se utiliza `ensure_ascii=False` para conservar caracteres como á, é, í, ó, ú y ñ de manera legible. Esta decisión es importante porque uno de los requisitos del laboratorio consiste en evidenciar que los caracteres especiales no se corrompen.

Para demostrar el funcionamiento, se utilizó un nombre de usuario que contiene tilde y ñ. El valor se conserva al guardar el JSON y vuelve a mostrarse correctamente después de reiniciar la aplicación.



12. Gestión de la foto de perfil

La selección de la foto se realiza desde el sistema de archivos mediante `QFileDialog`. La imagen elegida se copia a la carpeta `images` del proyecto y el JSON conserva una ruta relativa, evitando depender de una ubicación específica de la computadora del estudiante.

```
carpeta_imagenes = "images"

if not os.path.exists(carpeta_imagenes):
    os.makedirs(carpeta_imagenes)

extension = os.path.splitext(ruta)[1]
nombre_foto = "perfil" + extension

ruta_destino = os.path.join(carpeta_imagenes, nombre_foto)

shutil.copy2(ruta, ruta_destino)
self.ruta_foto = ruta_destino
```

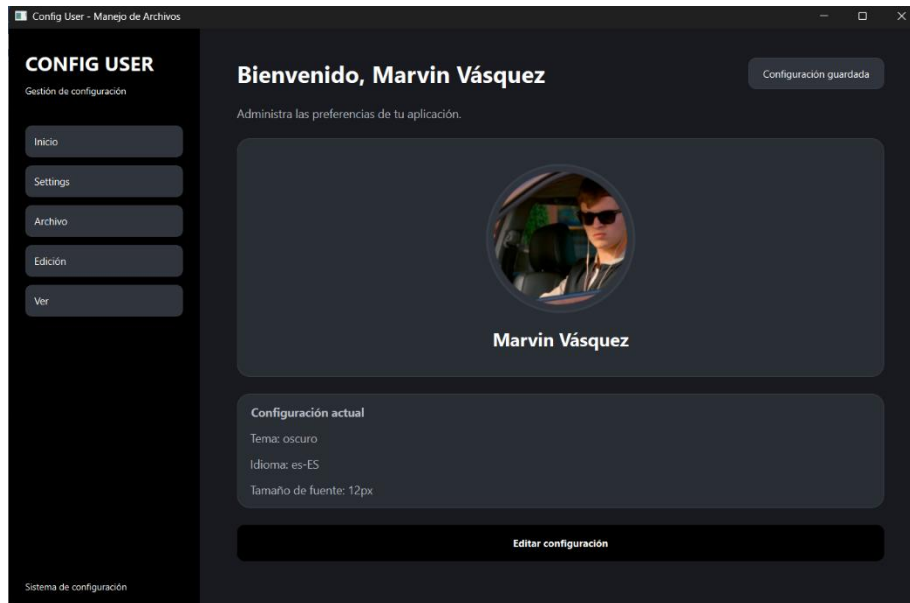
Al iniciar la aplicación, la ruta almacenada se utiliza para cargar la imagen y mostrarla en la ventana principal. Al cambiar la fotografía, el mismo flujo actualiza la configuración persistente.

13. Evidencias de funcionamiento

Esta sección reúne las capturas que deben acompañar el documento. El enunciado solicita evidencia de los tres casos de error y capturas generales de funcionamiento. Las imágenes deben corresponder a las pruebas realmente realizadas sobre la aplicación.

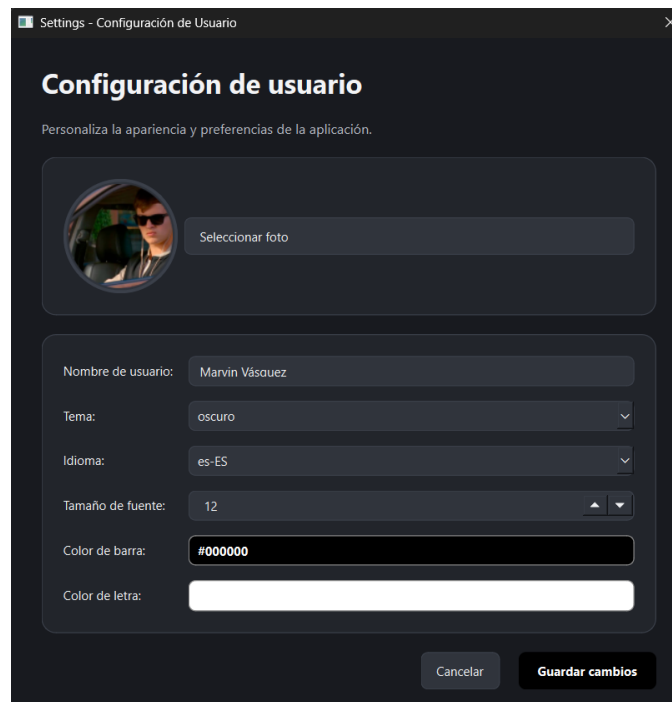
13.1. Menú principal

Captura de la ventana principal mostrando los cuatro menús.



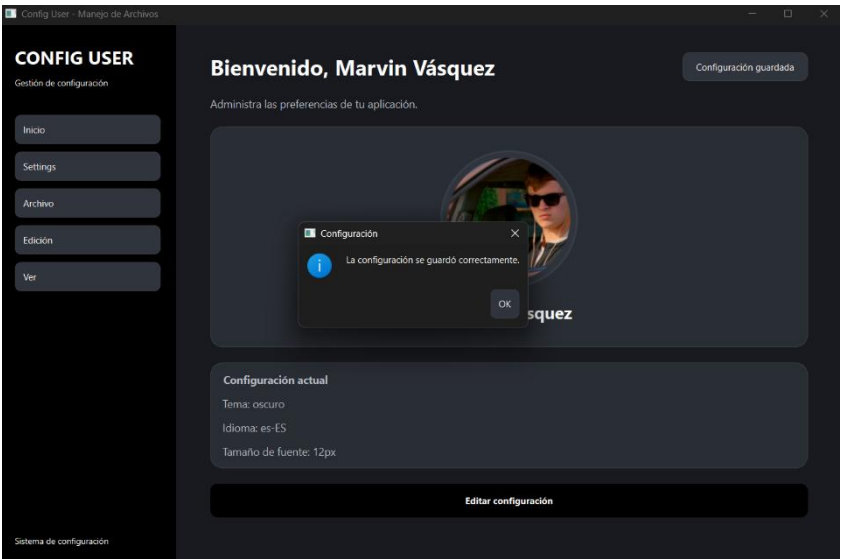
13.2. Settings

Captura con los campos, colores, fuente y selector de foto.



13.3. Persistencia

Captura de una configuración guardada y recuperada después de reiniciar.



14. Matriz de cumplimiento

Criterio del laboratorio	Implementación	Evidencia sugerida
Menú de aplicación con la estructura solicitada	Archivo, Edición, Ver y Settings.	Captura de ventana principal.
Ventana de Settings funcional	Permite modificar los siete parámetros.	Captura de Settings y prueba de guardado.
Carga y guardado	Carga JSON, valida y persiste cambios.	config.json + prueba de reinicio.
Errores + UTF-8	Archivo ausente, corrupto, inválido, permisos y UTF-8.	Capturas/logs de las pruebas.
Escritura y respaldo	config.bak + config.tmp + os.replace().	Captura y ejemplo de config.bak.
Comparación de formatos	JSON comparado con CSV.	Sección 5 del documento.
Presentación	Proyecto modular y explicable por el estudiante.	Defensa y demostración en clase.

15. Conclusiones

La solución desarrollada demuestra que el manejo de archivos puede mantenerse separado de la interfaz gráfica mediante un módulo de persistencia independiente. Esta organización facilita la lectura, validación, actualización y recuperación de los datos sin mezclar la lógica de archivos con la presentación visual.

La elección de JSON resulta coherente con la naturaleza de la información, ya que permite representar las preferencias como propiedades clave-valor y procesarlas directamente desde Python. La comparación con CSV muestra que, aunque ambos formatos son legibles y compactos, JSON se adapta de manera más directa a una configuración heterogénea.

La escritura segura, el respaldo y el manejo controlado de errores reducen el riesgo de pérdida o corrupción de la configuración. El uso explícito de UTF-8 garantiza que los datos con tildes y ñ se conserven correctamente, mientras que el almacenamiento de la foto dentro del proyecto mejora la portabilidad de la aplicación.

Finalmente, las pruebas y el historial incremental de Git permiten demostrar no solo el funcionamiento final, sino también la evolución y las decisiones técnicas tomadas durante el desarrollo.